基础课

- 排序
  - 快速排序——分治
  - 归并排序——分治
- 二分
  - 整数二分
  - 二分答案模板
  - 浮点数二分
  - P2249 【深基13.例1】查找
  - P1102 A-B 数对
- 高精度
  - vector
  - A+B
  - A-B
  - A*b
  - A ÷ b
- 前缀和与差分
  - 前缀和
    - 一维前缀
    - acwing795
      - python
      - cpp
    - 二维前缀
    - acwing 796
      - python
      - cpp
  - 差分
    - 一维差分
    - acwing 797
      - python
      - cpp
    - 二维差分
    - acwing 798
      - python
      - cpp
  - P1314 [NOIP 2011 提高组] 聪明的质监员
  - P2367 语文成绩
- 双指针算法
- 位运算
- 离散化
  - cpp
- 区间合并

提高课

- 位运算
  - AcWing 90. 64位整数乘法

基础课

数据输入>1000000时使用`scanf`，否则使用`cin`。

```
ios::sync_with_stdio(0);
cin.tie(0);             //cpp speed up
```

排序

排序算法	平均时间复杂度
冒泡排序	O(n2)
选择排序	O(n2)
插入排序	O(n2)
希尔排序	O(n1.5)
快速排序	O(N*logN)
归并排序	O(N*logN)
堆排序	O(N*logN)
基数排序	O(d(n+r))

快速排序——分治

1. 确定分界点
2. 调整区间
3. 递归处理左右两端

```
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int n;
int q[N];

void quick_sort(int q[], int l, int r)
```

```
{
    if(l >= r)   return;
    //int x = [q(l + r +1) / 2], i = l - 1, j = r + 1;
    int x = q[l + r >> 1], i = l - 1, j = r + 1;        //  注意是向上取整,因为向下取整可能使得x取到q[l]
    while(i < j)
    {
        do i ++ ; while (q[i] < x);             //  不能取等号，因为若数据都相等则无限循环触发数组越界
        do j -- ; while (q[j] > x);             //   do while 保证i,j一定更新
        if (i < j)  swap(q[i], q[j]);           //可以使用if(i <= j) swap(q[i], q[j]);
    }
     quick_sort(q, l, j), quick_sort(q, j + 1, r);
    //quick_sort(q, l, i - 1), quick_sort(q, i, r);
}
//关于quick_sort()取i，j的问题，若为i则取右边界，若为j则取左边界


int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i ++ )  scanf("%d", &q[i]);

    quick_sort(q, 0, n-1);

    for (int i = 0; i < n; i ++ )   printf("%d ", q[i]);

    return 0;
}
```

`C++ sort()` 其实是小范围插入排序，大范围快速排序

归并排序——分治

1. 确定分界点：二分之一
2. 递归排序left,right
3. 归并——合二为一

```

#include <iostream>

using namespace std;

const int N = 1e6 + 10;
int n;
int q[N],tmp[N];

void merge_sort(int q[], int l, int r)
{
    if (l >= r) return;         //结束递归返回
```

```c

    int mid = l + r >> 1;           //通过mid调整边界

    merge_sort(q, l, mid), merge_sort(q, mid + 1 , r);          //递归，先比较最小的分组

    int k = 0, i = l, j = mid + 1;
    while(i <= mid && j <= r)
        if (q[i] <= q[j])   tmp[ k ++ ] = q[ i ++ ];        //比较两部分
        else tmp[ k ++ ] = q [ j ++ ];
    while (i <= mid) tmp[ k ++ ] = q[ i ++ ];           //将剩余值写入，
    while (j <= r) tmp[ k ++ ] = q[ j ++ ]; ·           //由于前面的循环语句，这里最多只有一个while进行多次循环
    for (i = l, j = 0; i <= r; i ++, j ++)  q[i] =  tmp[j];     //写回数组
}
int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i ++)    scanf("%d", &q[i]);

    merge_sort(q, 0 , n-1);

    for (int i = 0; i < n; i++) printf("%d ", q[i]);

    return 0;

}
```

二分

整数二分

1. mid = (l+r+1)/2　找最后一个小于等于目标值的元素。（右边界）

- true　[mid, r]　l = mid
- false　[l, mid - 1]　r = mid -1

2. mid = (1+r)/2　查找第一个大于等于目标值的元素。（左边界）

- true　[l, mid]　r = mid
- false　[mid+1, r]　l = mid +1

二分答案模板

```
 while (left <= right) {
       int mid = left + (right - left) / 2;  // 计算中间值
       if (check(mid)) {  // 检查 mid 是否满足条件
           ans = mid;     // 更新答案
           right = mid - 1;  // 尝试更小的值
       } else {
           left = mid + 1;  // 尝试更大的值
       }
   }
```

```
//内置二分函数
int x = lower_bound(a, a + 8, 4) - a;   //[a,a+8) 查找的值    >=    严格递增序列的插入位置
int y = upper_bound(a, a + 8, 4) - a;   //                    >     非严格递增序列的插入位置
binary_search(start, end ,target):    //若目标值存在则返回true，否则返回false
```

```
#include <iostream>

using namespace std;

const int N = 100010;

int n, m;
int q[N];


int main()
{
    scanf("%d %d", &n, &m);
    for (int i = 0; i < n; i ++)      scanf("%d", & q[i]);
    while( m -- )
    {
        int x;
        scanf("%d", &x);

        int l = 0, r = n - 1;
        while (l < r)
        {
            int mid = l + r >> 1;               //左半二分，确定左边界
            if(q[mid] >= x) r = mid;
            else l = mid + 1;
        }
        if( q[l] != x) cout << "-1 -1" << endl;
```

```
        else
        {
            cout << l << ' ';
            int l = 0, r = n - 1;
            while (l < r)
            {
                int mid = l + r +1 >> 1;        //右半二分，确定右边界
                if(q[mid] <= x) l = mid;
                else r = mid - 1;
            }
            cout << l << endl;
        }
    }
    return 0;
}
```

浮点数二分

浮点数二分条件：区间长度大于精度。

```
#include <iostream>
using namespace std;

int main()
{
    double x;
    cin >> x;
    double l = -10000, r = 10000;           //l和r表示n的范围
    while (r - l > 1e-8)        //一般精度比题目+2次方
        {
            double mid = (l + r) / 2;       //二分
            if( mid * mid * mid >= x) r = mid;
            else l = mid;
        }
     printf("%lf\n",l);
    return 0;
}
```

P2249 【深基13.例1】查找

简单的二分查找并输出位置

```
#include <iostream>
#include <utility>
#include <algorithm>
```

```
using namespace std;
typedef long long LL;
const int N = 1e6 +10;
int arr[N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;

int main() {
    cin >> n >> m;
    int x;
    for(int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    for(int i = 0; i < m; i++)
    {
        cin >> x;
        int pos = lower_bound(arr,arr+n,x) - arr;

        if (pos < n && arr[pos] == x) {
            cout << pos + 1 << " ";
        } else {
            cout << -1 << " ";
        }
    }

}

```

P1102 A-B 数对

使用algorithm库中的lower_bound和upper_bound查找计算区间大小

```
#include <iostream>
#include <utility>
#include <algorithm>
using namespace std;
typedef long long LL;
const int N = 2e5 +10;

int arr[N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;
int max = 0;

long long sum = 0;
void solve() {
```

```
    sort(arr,arr+n);
    int tmp = -111;
    int cnt = 0;
    for(int i = 0; i <n; i++ )
    {
      //  cout << tmp << endl;
        if(arr[i] == tmp)
          {
            sum+=cnt;
            continue;
          }
        int x = arr[i] +m;
        if(binary_search(arr,arr+n,x))
            {
                int pos1 = lower_bound(arr,arr+n,x) - arr;
                int pos2 = upper_bound(arr,arr+n,x) - arr;
               // cout << arr[i] <<" " <<pos1 << " " << pos2<<endl;
                cnt = pos2 - pos1;
                sum+= cnt;
                tmp = arr[i];
            }
    }


}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    solve();
    cout << sum << endl;
}

```

高精度

A+B、A-B、A*b、A÷b，直接打板子

在`Java`中，可以使用`BigInteger`类来表示任意精度的整数，以及`BigDecimal`类来表示任意精度的浮点数。

在`Python`中，可以直接使用内置的`int`类型来表示任意精度的整数。Python的整数类型可以自动扩展以适应大数值，不会发生溢出。

vector

`向量（Vector）`是一个封装了动态大小数组的顺序容器（Sequence Container）。跟任意其它类型容器一样，它能够存放各种类型的对象。可以简单的认为，向量是一个能够存放任意类型的动态数组。

1.顺序序列

顺序容器中的元素按照严格的线性顺序排序。可以通过元素在序列中的位置访问对应的元素。

2.动态数组

支持对序列中的任意元素进行快速直接访问，甚至可以通过指针算述进行该操作。提供了在序列末尾相对快速地添加/删除元素的操作。

3.能够感知内存分配器的（Allocator-aware）

容器使用一个内存分配器对象来动态地处理它的存储需求。

[C++ vector 容器浅析 | 菜鸟教程 (runoob.com)]

A+B

1. 大整数存储，数组起始位置存放最低位置
2. 按照数学加法原理进行计算

```
#include <iostream>
#include <vector>

using namespace std;

vector<int> add(vector<int> &A, vector<int> &B)
{
        vector<int> C;
        int t = 0;
        if (A.size() < B.size())   return add(B, A);
        for (int i = 0; i < A.size() || i < B.size(); i ++ )
        {
            //t + = A[i];
            if (i < A.size())   t += A[i];
            if (i < B.size())   t += B[i];
            C.push_back(t % 10);
            t /= 10;            //去进位
        }
        if (t) C.push_back(1);      //有无进位判断
        return C;

}
```

```


int main()
{
    string a,b;
    vector<int> A, B;
    cin >> a >> b ;
    for (int i = a.size() - 1; i >= 0; i --)    A.push_back(a[i] - '0');
    //push_back在数组的最后添加一个数据,类似于栈
    for (int i = b.size() - 1; i >= 0; i --)    B.push_back(b[i] - '0');

    auto C = add(A, B);

    for (int i = C.size() - 1; i >= 0; i -- )   printf("%d", C[i]);

    return 0;

}
```

A-B

1. 大整数存储，数组起始位置存放最低位置
2. 判断A，B大小，大数减小数，加符号
3. Ai - Bi 借位
4. 若为符号，转化为绝对值相加减

```
#include <iostream>
#include <vector>

using namespace std;

bool cmp(vector<int> &A, vector<int> &B)
{
        if(A.size() != B.size())    return A.size() > B.size();
        for (int i = A.size() - 1; i >= 0; i -- )
            if (A[i] != B[i])
                return A[i] > B[i];

        return true;
}


vector<int> sub(vector<int> &A, vector<int> &B)
{
        vector<int> C;


        for (int i = 0 ,t = 0; i < A.size(); i++)
           {
            t = A[i] - t;               //消除进位
```

```
            if( i < B.size())   t -= B[i];      //运算
            C.push_back((t + 10) % 10);         //求绝对值
            if ( t < 0) t = 1;          //产生进位
            else
                t = 0;
            }


        while (C.size() > 1 && C.back() == 0)   C.pop_back();   //去除多余的0

        return C;

}


int main()
{
    string a,b;
    vector<int> A, B;
    cin >> a >> b ;
    for (int i = a.size() - 1; i >= 0; i --)    A.push_back(a[i] - '0');
    //push_back在数组的最后添加一个数据,类似于栈
    for (int i = b.size() - 1; i >= 0; i --)    B.push_back(b[i] - '0');

    if (cmp(A, B))          //大减小
          {
            auto C = sub(A, B);
            for (int i = C.size() - 1; i >= 0; i -- )   printf("%d", C[i]);
         }
    else
          {
            auto C = sub(B, A);
            printf("-");
            for (int i = C.size() - 1; i >= 0; i -- )   printf("%d", C[i]);
         }
    return 0;

}
```

A*b

```
#include <iostream>
#include <vector>

using namespace std;

 vector<int> mul(vector<int> &A, int b)
 {
        vector<int> C;
        int t = 0;
        for (int i = 0; i < A.size() || t; i ++ )
        {
```

```
            if (i < A.size())   t += A[i] * b;
            C.push_back(t % 10);
            t /= 10;
        }
     while (C.size() > 1 && C.back() == 0) C.pop_back();            //去除0

    return C;
 }

int main()
{
    string a;
    int b;
    cin >> a >> b;
    if (b == 0) {printf("0");   return 0;}      //0特解

    vector<int> A;

    for (int i = a.size() - 1; i >= 0; i -- )   A.push_back(a[i] - '0');

    auto C = mul(A, b);

    for (int i = C.size() - 1; i >= 0; i --)     printf("%d",C[i]);

    return 0;
}
```

A ÷ b

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

 vector<int> div(vector<int> &A, int b, int &r)
 {
        vector<int> C;
        r = 0;

        for (int i = A.size() - 1; i >= 0 ; i -- )
        {
            r = r * 10 + A[i];
            C.push_back(r/b);
            r %= b;
        }
        reverse(C.begin(), C.end());
     while (C.size() > 1 && C.back() == 0) C.pop_back();            //去除0

    return C;
 }

```

```
int main()
{
    string a;
    int b;
    cin >> a >> b;

    vector<int> A;

    for (int i = a.size() - 1; i >= 0; i -- )   A.push_back(a[i] - '0');
    int r;
    auto C = div(A, b, r);

    for (int i = C.size() - 1; i >= 0; i --)     printf("%d",C[i]);
    cout << endl << r <<endl;
    return 0;
}

```

前缀和与差分

区间更新 + 区间查询：先用差分处理区间更新，再用前缀和还原数组并查询区间和。

> 前缀和和差分的细节差异
>
> 前缀和　s[i] = s[i-1] + a[i]
>
> sum = s[r] - s[l-1]
>
> 差分　d[i] = a[i] - a[i-1]　(i > 0)
> 增加val
>
> d[l] += val
> d[r+1] -= val

前缀和

一维前缀

- 一次运算计算出所有区间的和，空间换时间
- 快速计算任意区间的和。

acwing795

输入一个长度为 n 的整数序列。

接下来再输入 m 个询问，每个询问输入一对 l, r。

对于每个询问，输出原序列中从第 l 个数到第 r 个数的和。

输入格式

第一行包含两个整数 n 和 m。

第二行包含 n 个整数，表示整数数列。

接下来 m 行，每行包含两个整数 l 和 r，表示一个询问的区间范围。

输出格式

共 m 行，每行输出一个询问的结果。

数据范围

$1 \leq l \leq r \leq n,$
$1 \leq n, m \leq 100000,$
$-1000 \leq \text{数列中元素的值} \leq 1000$

输入样例：

```
5 3
2 1 3 6 4
1 2
1 3
2 4
```

输出样例：

```
3
6
10
```

python

```python
import sys
import os
input = lambda: sys.stdin.readline().strip()
sys.setrecursionlimit(1000000)
n,m = map(int,input().split())


s = list(map(int,input().split()))

brr = [0]*(n+10)

for i in range(n):
    brr[i+1] = brr[i] + s[i]


for i in range(m):
```

```
    a,b = map(int,input().split())
    print(brr[b] - brr[a-1])

```

cpp

```cpp
#include <iostream>

using namespace std;

const int N = 100010;
int n, m;
int a[N], s[N];


int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i ++ )  scanf("%d", &a[i]);

    for (int i = 1; i <= n; i ++ )  s[i] = s[i - 1] +a[i];

    while( m-- )
    {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%d\n",s[r]- s[l -1 ]);
    }
    return 0;

}
```

二维前缀

acwing 796

输入一个 n 行 m 列的整数矩阵，再输入 q 个询问，每个询问包含四个整数 x_1, y_1, x_2, y_2，表示一个子矩阵的左上角坐标和右下角坐标。

对于每个询问输出子矩阵中所有数的和。

输入格式

第一行包含三个整数 n，m，q。

接下来 n 行，每行包含 m 个整数，表示整数矩阵。

接下来 q 行，每行包含四个整数 x_1, y_1, x_2, y_2，表示一组询问。

输出格式

共 q 行，每行输出一个询问的结果。

数据范围

$1 \le n, m \le 1000$,
$1 \le q \le 200000$,
$1 \le x_1 \le x_2 \le n$,
$1 \le y_1 \le y_2 \le m$,
$-1000 \le$ 矩阵内元素的值 ≤ 1000

输入样例：

```
3 4 3
1 7 2 4
3 6 2 8
2 1 2 3
1 1 2 2
2 1 3 4
1 3 3 4
```

输出样例：

```
17
27
21
```

python

```python
n,m,q = map(int,input().split())

arr = []

for _ in range(n):
    arr.append(list(map(int,input().split())))

brr = [[0]*(m+10) for _ in range(n+10)]


for i in range(n):
    for j in range(m):
        brr[i+1][j+1] = brr[i+1][j] + brr[i][j+1] -brr[i][j] + arr[i][j]
```

```python

for _ in range(q):
    x1,y1,x2,y2 = map(int,input().split())
    print(brr[x2][y2] - brr[x1-1][y2] - brr[x2][y1-1] + brr[x1-1][y1-1])

```

cpp

- 重点在于减去交叉区间的重复计算

```cpp
#include <iostream>

using namespace std;

const int N = 1010;
int n, m, q;
int a[N][N], s[N][N];


int main()
{


    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
             scanf("%d", &a[i][j]);

 for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
            s[i][j] = s[i - 1][j] + s[i][ j - 1] - s[i - 1][j - 1] + a[i][j];
            //计算前缀和

    while( q-- )
    {
        int x1, x2, y1, y2;
        scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        printf("%d\n",s[x2][y2] - s[x1 -1][y2] - s[x2][y1 - 1] + s[x1 -1][y1 - 1]);
        //计算子矩阵的和
    }
    return 0;

}
```

差分

- 前缀和的逆运算
- 数组b[i]即原数组a[1]+.....+a[i]
- 用O(1)的时间给原数组某个区间内的值全部加上C
- 用于高效处理区间更新问题。

a[0]= 0;

b[1] = a[1] - a[0];

b[2] = a[2] - a[1];

b[3] =a [3] - a[2];

.....

b[n] = a[n] - a[n-1];

一维差分

acwing 797

输入一个长度为 n 的整数序列。

接下来输入 m 个操作，每个操作包含三个整数 l, r, c，表示将序列中 $[l, r]$ 之间的每个数加上 c。

请你输出进行完所有操作后的序列。

输入格式

第一行包含两个整数 n 和 m。

第二行包含 n 个整数，表示整数序列。

接下来 m 行，每行包含三个整数 l，r，c，表示一个操作。

输出格式

共一行，包含 n 个整数，表示最终序列。

数据范围

$1 \le n, m \le 100000,$
$1 \le l \le r \le n,$
$-1000 \le c \le 1000,$
$-1000 \le$ 整数序列中元素的值 ≤ 1000

输入样例：

```
6 3
1 2 2 1 2 1
1 3 1
3 5 1
1 6 1
```

输出样例：

```
3 4 5 3 4 2
```

python

```python
import sys
sys.setrecursionlimit(10000)
input = lambda : sys.stdin.readline().strip()
n, m = map(int, input().split())

arr = [0]+ list(map(int, input().split()))

brr = [0] * (n + 10)

for i in range(1,n+1):
    brr[i] = arr[i] - arr[i-1]

for _ in range(m):
    a,b,c = map(int, input().split())
    brr[a] += c
    brr[b+1] -= c

for i in range(1, n+1):
```

```python
    arr[i] = brr[i] + arr[i-1]

for i in range(1, n+1):
    print(arr[i], end=' ')
```

cpp

```cpp
#include <iostream>

using namespace std;

const int N = 100010;

int n, m;
int a[N], b[N];     //a[N]原矩阵，b[N]差分矩阵

void insert(int l, int r, int c)
{
    b[l] += c;
    b[r+1] -= c;
}

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i ++) scanf("%d", &a[i]);

    for (int i = 1; i <= n; i ++)   insert(i, i ,a[i]);

    while( m -- )
        {
            int l, r , c;
            scanf("%d%d%d", &l, &r, &c);
            insert(l, r, c);
        }
    for (int i = 1; i <= n; i++) b[i] += b[i - 1];

    for (int i = 1; i <= n; i ++)   printf("%d ",b[i]);

    return 0;


}

```

二维差分

acwing 798

输入一个 n 行 m 列的整数矩阵，再输入 q 个操作，每个操作包含五个整数 x_1, y_1, x_2, y_2, c，其中 (x_1, y_1) 和 (x_2, y_2) 表示一个子矩阵的左上角坐标和右下角坐标。

每个操作都要将选中的子矩阵中的每个元素的值加上 c。

请你将进行完所有操作后的矩阵输出。

输入格式

第一行包含整数 n, m, q。

接下来 n 行，每行包含 m 个整数，表示整数矩阵。

接下来 q 行，每行包含 5 个整数 x_1, y_1, x_2, y_2, c，表示一个操作。

输出格式

共 n 行，每行 m 个整数，表示所有操作进行完毕后的最终矩阵。

数据范围

$1 \le n, m \le 1000$,
$1 \le q \le 100000$,
$1 \le x_1 \le x_2 \le n$,
$1 \le y_1 \le y_2 \le m$,
$-1000 \le c \le 1000$,
$-1000 \le$ 矩阵内元素的值 ≤ 1000

输入样例：

```
3 4 3
1 2 2 1
3 2 2 1
1 1 1 1
1 1 2 2 1
1 3 2 3 2
3 1 3 4 1
```

输出样例：

```
2 3 4 1
4 3 4 1
2 2 2 2
```

python

```python
import sys
import os
sys.setrecursionlimit(100000)
input =  lambda: sys.stdin.readline().strip()

arr = []

n,m,q = map(int,input().split())
```

```python

arr.append([0]*(m+10))

for _ in range(n):
    arr.append([0] + list(map(int,input().split())))

brr = [[0]*(m+10) for _ in range(n+10)]

for i in range(1,n+1):
    for j in range(1,m+1):
        brr[i][j] = arr[i][j] - arr[i][j-1] - arr[i-1][j] + arr[i-1][j-1]




for _ in range(q):
    x1,y1,x2,y2,c = map(int,input().split())
# 对矩形区域(x1,y1)到(x2,y2)增加c的差分操作
    brr[x1][y1] += c          # 影响所有右下区域
    brr[x2+1][y1] -= c        # 取消右侧影响
    brr[x1][y2+1] -= c        # 取消下侧影响
    brr[x2+1][y2+1] += c      # 补回重复减去的区域

for i in range(1,n+1):
    for j in range(1,m+1):
        arr[i][j] = brr[i][j] + arr[i][j-1] + arr[i-1][j] - arr[i-1][j-1]

for i in range(1, n+1):
        print(' '.join(map(str, arr[i][1:m+1])))
```

cpp

```cpp
#include <iostream>


using namespace std;

const int N = 1010;
int n, m ,q;
int a[N][N], b[N][N];


void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    b[x2 + 1][y1] -= c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y2 + 1] += c;
}

int main()
```

```
{
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
             scanf("%d", &a[i][j]);

     for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
            insert(i, j, i ,j ,a[i][j]);

   while(q -- )
    {
        int x1, x2, y1, y2, c;
        cin >> x1 >> y1 >> x2 >> y2 >>c;
        insert(x1, y1, x2 ,y2 ,c);
    }

     for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
            b[i][j] += b[i - 1][j] + b[i][j - 1] -b[i - 1][j - 1];  //逆运算

     for (int i = 1; i <= n; i ++ )
        {
            for (int j = 1; j <= m; j ++ )  printf("%d", b[i][j]);
            puts("");


    return 0;
}
```

P1314 [NOIP 2011 提高组] 聪明的质监员

一个结合前缀和和二分答案的好题。虽然这题思路还是较为简单的，但是我做了整整一个多小时。整理下这题收获。

1. 二分答案中check函数应只写判断，简化二分判断逻辑
2. 注意初始化二分上下限，否则能卡半个小时
3. 计算前缀和时，必须加上该层循环的值或者令s[i]，s[i-1]，使得前缀和成立（不然就是区间和）

```
#include <iostream>
#include <utility>
#include <algorithm>
using namespace std;
typedef long long LL;
const long long N = 2e5 +10;
long long arr[N][2];
long long query[N][2];

```

```
typedef pair<long long, long long> PA;
typedef pair<PA, long long> PA2;
long long n,m,q;

long long ans = 1e13;
bool solve(long long w) {
    long long brr[N] = {0};
    long long crr[N] = {0};
    long long  sum = 0;
    for(long long i = 1; i<= n; i++)
    {
        if(arr[i][0]>=w)
            {
                brr[i] = brr[i-1]+arr[i][1];
                crr[i] = crr[i-1] +1;
            }
        else
           {
            brr[i] = brr[i-1];      //前缀和必须继承
            crr[i] = crr[i-1];
           }
    }
    for(long long i = 1; i<= m; i++)
        {
            sum += (brr[query[i][1]] - brr[query[i][0]-1])*  (crr[query[i][1]] - crr[query[i][0]-1]);
        }
    ans = min(ans,llabs(sum-q));
    return sum >q;

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m >> q;
    for(long long i = 1; i<= n; i++)
    {
        cin>> arr[i][0] >> arr[i][1];
    }
    for(long long i = 1; i<= m; i++)
    {
        cin >> query[i][0] >> query[i][1];
    }


    long long l = -1;
    long long r = 2e10+10;

    while(l<=r)
        {
            long long mid = l + ((r-l) >> 1);
            if(!solve(mid)){
```

```
                r = mid -1;
            }else {
                l = mid+1;
            }

        }

    cout << ans << endl;

}

```

P2367 语文成绩

复制 Markdown 折叠 进入 IDE 模式

题目背景

语文考试结束了，成绩还是一如既往地有问题。

题目描述

语文老师总是写错成绩，所以当她修改成绩的时候，总是累得不行。她总是要一遍遍地给某些同学增加分数，又要注意最低分是多少。你能帮帮她吗？

输入格式

第一行有两个整数 n，p，代表学生数与增加分数的次数。

第二行有 n 个数，$a_1 \sim a_n$，代表各个学生的初始成绩。

接下来 p 行，每行有三个数，x，y，z，代表给第 x 个到第 y 个学生每人增加 z 分。

输出格式

输出仅一行，代表更改分数后，全班的最低分。

输入输出样例

输入 #1 复制

```
3 2
1 1 1
1 2 1
2 3 1
```

输出 #1 复制

```
2
```

说明/提示

对于 40% 的数据，有 $n \le 10^3$。

对于 60% 的数据，有 $n \le 10^4$。

对于 80% 的数据，有 $n \le 10^5$。

对于 100% 的数据，有 $n \le 5 \times 10^6$，$p \le n$，学生初始成绩 ≤ 100，$z \le 100$。

基本的差分计算和复原。

```
#include <iostream>
#include <algorithm>
using namespace std;
typedef long long LL;
const int N = 5e6 +10;
```

```
int arr[N];
int brr[N];

int n,p;


void solve() {
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> p;
    for(int i = 1; i<=n;i++)
    {
        cin >> arr[i];
    }

    for(int i = 1; i <= n; i++)
    {
        brr[i] = arr[i] - arr[i-1]; //求差分数组
    }

    for(int i = 1; i <= p; i++)
    {
        int x,y,z;
        cin >> x >> y >> z;
        brr[x]+= z;             //差分左区间更新
        brr[y+1] -= z;          //差分右区间更新
    }
    for(int i = 1; i <= n; i++)
    {
        arr[i] = brr[i] +arr[i-1];  //复原
    }
    sort(arr+1,arr+1+n);

    cout << arr[1] << endl;
}

```

双指针算法

- 将O(n**2)优化到O(n)

for (int i = 0, j = 0; i < n; i ++)
{
  while (j < i && check(i, j)) j ++ ;

}

for (int i = 1; i <= n; i ++)
    for (int j = 1; j <= m; j ++)

```
#include <iostream>
using namespace std;

const int N = 100010;
int a[N], s[N]; //s[N]统计字母出现次数
//j指向不重复序列头，i指向不重复序列尾
int main()
{
    int n, r = 0;
    cin >> n;
    for (int i = 0, j = 0; i < n; i++)
    {
        cin >> a[i];
        ++ s[a[i]];
        while (s[a[i]] > 1) -- s[a[j++]];   //不能用if，因为当检测到两个相同字母时，要j++直到消除先添加的字母。
        r = max(r, i - j + 1);
    }
    cout << r;
    return 0;
}
```

位运算

lowbit(x)

```
#include <iostream>
using namespace std;

int lowbit(int x)
{
    return x & -x;          //10010 &   (01101+1)
}

int main()
{
    int n;
    cin >> n;
    while(n --)
    {
        int x;
        cin >> x;
        int res = 0;
       while(x)    x-= lowbit(x), res++;

       cout << res << ' ';
    }
```

```
    return 0;
}
```

直观方法，每位判断【不通过除以x（除法速度慢），而是通过右移1加快运算速度】

```
int count_ones(int x) {
    int count = 0;
    for (int i = 0; i < 32; i++) { // 假设int为32位
        if (x & (1 << i)) {        // 检查第i位是否为1
            count++;
        }
    }
    return count;
}
```

离散化

值域跨度大，但是很稀疏

cpp

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

typedef pair<int, int>  PII;

const int N = 300010;

int n, m;
int a[N], s[N];


vector<int> alls;
vector<PII> add, query;


int find(int x)         //二分查找
{
    int l = 0, r = alls.size() - 1;
    while(l < r)
```

```
    {
        int mid = l + r >> 1;
        if(alls[mid] >= x)  r = mid;
        else l = mid + 1;
    }
    return r + 1;
}

int main()
{
    cin >> n >> m;
    for (int i = 0;i < n; i ++)
    {
        int x, c;
        cin >> x >> c;
        add.push_back({x,c});
        alls.push_back(x);
    }

    for (int i = 0;i < m; i ++)
    {
        int l, r;
        cin >> l >> r;
        query.push_back({l, r});        //存储查找键值
        alls.push_back(l);
        alls.push_back(r);
    }

    //去重,值可以累加，但是坐标只有一个
    sort(alls.begin(), alls.end());
    alls.erase(unique(alls.begin() ,alls.end()), alls.end()) ;

    for (auto item : add)   //迭代器遍历
    {
        int x = find(item.first);
        a[x] += item.second;
    }

    for (int i = 1 ; i <= alls.size(); i ++)    s[i] = s[i - 1] + a[i]; // 区间和

    for (auto item : query)
    {
        int l = find(item.first),   r = find(item.second);
        cout << s[r] - s[l - 1] << endl;
    }

    return 0;
}
```

区间合并

- 按区间左端点排序

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

typedef pair<int, int>  PII;

const int N = 300010;

int n, m;
int a[N], s[N];


vector<int> alls;
vector<PII> add, query;


int find(int x)         //二分查找
{
    int l = 0, r = alls.size() - 1;
    while(l < r)
    {
        int mid = l + r >> 1;
        if(alls[mid] >= x)  r = mid;
        else l = mid + 1;
    }
    return r + 1;
}

int main()
{
    cin >> n >> m;
    for (int i = 0;i < n; i ++)
    {
        int x, c;
        cin >> x >> c;
        add.push_back({x,c});
        alls.push_back(x);
    }

    for (int i = 0;i < m; i ++)
    {
        int l, r;
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l);
        alls.push_back(r);
    }

```

```

    sort(alls.begin(), alls.end());
    alls.erase(unique(alls.begin() ,alls.end()), alls.end()) ;

    for (auto item : add)
    {
        int x = find(item.first);
        a[x] += item.second;
    }

    for (int i = 1 ; i <= alls.size(); i ++)    s[i] = s[i - 1] + a[i];

    for (auto item : query)
    {
        int l = find(item.first),   r = find(item.second);
        cout << s[r] - s[l - 1] << endl;
    }

    return 0;
}

```

提高课

位运算

AcWing 90. 64位整数乘法

90. 64位整数乘法

题目 提交记录 讨论 题解 视频讲解

求 a 乘 b 对 p 取模的值。

输入格式

第一行输入整数a，第二行输入整数b，第三行输入整数p。

输出格式

输出一个整数，表示 `a*b mod p` 的值。

数据范围

$1 \le a, b, p \le 10^{18}$

输入样例：

```
3
4
5
```

输出样例：

```
2
```

- 将 b分解为 二进制形式
- 用倍增法计算

```cpp
#include <iostream>
#include <cstdio>
#define ll long long
using namespace std;
int main()
{
    ll a,b,p,res;
    cin>>a>>b>>p;
    res=0;
    while(b)
    {
        if(b&1)
            res=(res+a)%p;
        b>>=1;
        a=2*a%p;
    }
    cout<<res<<endl;
    return 0;
}

```